

CampusPaddy AI Chatbot Technical Implementation Document

Teesside University Knowledge Retrieval System

Anthony Efemena Edjenuwa
Teesside University

May 6, 2026

Executive Summary

CampusPaddy is a production-ready AI-powered chatbot system designed to provide Teesside University students with instant access to university knowledge through intelligent retrieval-augmented generation (RAG). The system combines web scraping, natural language processing, and vector-based information retrieval to deliver contextually accurate answers.

1 System Architecture Overview

The CampusPaddy system follows a multi-tier architecture:

1. **Teesside Website** - Source of knowledge
2. **Scraper Server** - Web crawler and data processor
3. **Local PostgreSQL** - Development database
4. **Neon Cloud Database** - Production database
5. **AI Server** - REST API with LLM
6. **Cloudflare Tunnel** - Public access

2 Key Components

2.1 Scraper Server (campuspaddy-ai-importer)

Responsible for:

Discovering and crawling Teesside University website

Extracting content from HTML, PDF, and DOCX files

Chunking documents into manageable knowledge units

Storing processed data in PostgreSQL

2.2 AI Server (campuspaddy-retrieval-api)

Provides:

RESTful API endpoints for question answering

Full-text search with PostgreSQL tsvector

Caching layer for repeated questions

Local Ollama LLM integration

Performance optimizations

3 Database Schema

3.1 Universities Table

Stores metadata about universities (Teesside University):

```
1 CREATE TABLE universities (
2   id UUID PRIMARY KEY,
3   name TEXT NOT NULL,
4   domain TEXT NOT NULL UNIQUE,
5   created_at TIMESTAMP DEFAULT now()
6 );
```

3.2 Knowledge Sources

Represents extracted content from each source document:

```
1 CREATE TABLE knowledge_sources (
2   id UUID PRIMARY KEY,
3   university_id UUID REFERENCES universities(id),
4   url TEXT NOT NULL UNIQUE,
5   title TEXT,
6   source_type TEXT NOT NULL,
7   extracted_text TEXT,
8   status TEXT DEFAULT 'pending',
9   processed_at TIMESTAMP,
10  created_at TIMESTAMP DEFAULT now()
11 );
```

3.3 Knowledge Chunks

Stores chunks of processed text:

```

1 CREATE TABLE knowledge_chunks (
2   id UUID PRIMARY KEY,
3   source_id UUID REFERENCES knowledge_sources(id),
4   university_id UUID REFERENCES universities(id),
5   chunk_index INT NOT NULL,
6   chunk_text TEXT NOT NULL,
7   created_at TIMESTAMP DEFAULT now()
8 );

```

3.4 Cached Answers

Caches generated answers with 14-day TTL:

```

1 CREATE TABLE chatbot_cached_answers (
2   id UUID PRIMARY KEY,
3   university_domain TEXT NOT NULL,
4   normalized_question TEXT NOT NULL,
5   answer TEXT NOT NULL,
6   sources JSONB DEFAULT '[]',
7   hit_count INT DEFAULT 0,
8   expires_at TIMESTAMPTZ DEFAULT now() + interval '14 days',
9   created_at TIMESTAMPTZ DEFAULT now(),
10  UNIQUE (university_domain, normalized_question)
11 );

```

4 Web Scraping Methodology

4.1 Discovery Strategy

Multiple entry points for crawling:

- Core start URLs from main navigation

- Sitemap XML parsing

- Hyperlink discovery using BFS

- Maximum depth: 8 levels

- Maximum pages: 5000

4.2 Ethical Considerations

Respects robots.txt directives

Rate limiting: 1.5 seconds between requests

Identifies as educational bot in User-Agent

Blocks private portals and login endpoints

Skips authentication and payment systems

4.3 Content Types Supported

HTML pages (97.9% of sources)

PDF documents (3.9% of sources)

DOCX files (0.7% of sources)

Binary DOC files not supported

5 Data Processing Pipeline

Processing workflow:

1. URL discovery via crawling
2. Raw file storage to disk
3. Text extraction from files
4. Sentence-based chunking (1000 chars)
5. Database insertion with metadata
6. Priority scoring by URL patterns

5.1 Processing Statistics

Total sources processed: 11,753

Processing failures: 306 (2.6%)

Success rate: 97.4%

Total knowledge chunks: 25,345

6 Retrieval Augmented Generation (RAG)

6.1 Dual-Strategy Retrieval

1. **Primary:** Full-text search using tsvector
2. **Fallback:** Keyword matching on titles and content

6.2 Scoring Algorithm

Score combines multiple factors:

Chunk text relevance (weight: 1.0)

Source title relevance (weight: 2.5)

URL pattern boost (0.04 - 0.08)

Student-relevant pages get priority boost

6.3 Chunk Diversification

Maximum 2 chunks per source URL

Prevents redundant information

Improves context diversity

7 Large Language Model Integration

7.1 Ollama Configuration

Model: llama3.1:8b (8 billion parameters)

Context window: 4,096 tokens

Running locally on GPU (NVIDIA Tesla P40)

VRAM: 6,000-7,000 MB for model

Keep-alive: 24 hours

7.2 Model Parameters

Temperature: 0.2 (low for factual answers)

Top-p: 0.9 (nucleus sampling)

Max prediction tokens: 150

Timeout: 60 seconds

7.3 Response Generation

Context: Retrieved chunks provided as sources

System prompt: Guides bot behavior

User prompt: Student question

Output: Concise, sourced answer

8 Caching Strategy

8.1 Two-Tier Caching

1. In-memory cache (application level)
2. PostgreSQL persistent cache

8.2 Question Normalization

Lowercase conversion

Punctuation removal

Whitespace normalization

Enables cache hits across variations

8.3 Cache Performance

Cache hit: 200-700ms

Uncached request: 5.3s

Speedup: 7.5x for cached queries

TTL: 14 days

9 REST API Specification

9.1 Base URL

`https://campuspaddy-ai.edjenuwahome.uk`

9.2 Key Endpoints

GET /health - System health check

GET /ollama/health - Model availability

GET /cache/stats - Cache statistics

POST /retrieve - Get relevant chunks

POST /ask - Question answering

9.3 POST /ask Example

Request:

```
1 {  
2   "question": "How do I contact accommodation?",  
3   "limit": 3,  
4   "useCache": true,  
5   "universityDomain": "tees.ac.uk"  
6 }
```

Response:

```
1 {  
2   "ok": true,  
3   "cached": false,  
4   "answer": "You can contact accommodation...",  
5   "sources": [...],  
6   "duration_ms": 5300  
7 }
```

10 Performance Optimization

10.1 Before Optimization

Retrieval: 7-12 seconds

Ollama: 4-5 seconds

Total uncached: 13-16 seconds

10.2 After Optimization

Retrieval: 0.95 seconds (88% improvement)

Ollama: 4.3 seconds (consistent)

Total uncached: 5.3 seconds (68% improvement)

10.3 Optimization Techniques

1. Full-text search indexing with tsvector
2. Query optimization and early termination
3. Result limiting and diversification
4. Connection pooling
5. Model keep-alive in VRAM

11 Security Considerations

11.1 API Security

API key authentication via x-api-key header

Generate key: openssl rand -hex 32

Health endpoints public (no auth)

Ask endpoint requires authentication

11.2 Data Protection

Never include in submitted or shared materials:

.env files

Database URLs

API keys

Cloudflare credentials

Private certificates

11.3 Web Crawler Security

Respects robots.txt

Rate limiting prevents abuse

User-Agent identifies crawler

Blocks authentication endpoints

12 Deployment Architecture

12.1 Development

Local Linux with PostgreSQL

Data in ./storage directories

Adminer for database inspection

Manual testing with curl

12.2 Production

Linux VPS with Node.js

Express API on localhost:3011

Ollama on localhost:11434

Neon cloud PostgreSQL

Cloudflare Tunnel for public access

systemd service for auto-restart

12.3 systemd Service

Auto-start on boot

Auto-restart on failure

Logging to systemd journal

Status monitoring

13 Project Statistics

13.1 Knowledge Base

Total chunks: 25,345

Total sources: 11,753

Success rate: 97.4%

Average chunk size: 1,000 characters

Total uncompressed: 150 MB

13.2 Performance Metrics

P50 response time: 5.1s uncached

P95 response time: 5.8s uncached

Cache hit rate: 60-70%

Cached response: 0.2-0.7s

Average crawl duration: 6-8 hours

13.3 Processing Results

HTML pages: 11,503 (97.9%)

PDF documents: 454 (3.9%)

DOCX files: 80 (0.7%)

Binary DOC files: 22 (failed, not supported)

14 Technical Stack

14.1 Backend

Language: TypeScript

Runtime: Node.js

Framework: Express.js

Database: PostgreSQL

Scraper: Cheerio

PDF Parser: pdf-parse

DOCX Parser: mammoth

LLM Runtime: Ollama

14.2 Infrastructure

Development DB: PostgreSQL (local)

Production DB: Neon (serverless)

Compute: Linux VPS

Acceleration: NVIDIA GPU (Tesla P40)

Access: Cloudflare Tunnel

Service Manager: systemd

Version Control: Git

15 Future Improvements

15.1 Short-Term

Streaming responses (Server-Sent Events)

User feedback rating system

Query analytics and gap detection

Multi-model support

15.2 Medium-Term

Vector embeddings (pgvector)

Web UI for chatbot

PDF upload and query

Multi-university support

15.3 Long-Term

Model fine-tuning

Mobile app

RAG optimization techniques

Portal integration

16 Conclusion

CampusPaddy demonstrates a production-ready implementation of Retrieval-Augmented Generation applied to university student support. The system successfully combines responsible web scraping, scalable data processing, optimized retrieval, and LLM integration.

Key achievements:

Ethical web crawler respecting robots.txt

25,000+ processed knowledge chunks
88% retrieval performance improvement
7.5x speedup through caching
Production deployment with high availability

17 Submission Package

The submitted package contains the source code, technical documentation, and deployment templates needed to review the implementation.

Main report: docs/reports/TECHNICAL_IMPLEMENTATION.pdf

Source report: docs/reports/TECHNICAL_IMPLEMENTATION.tex

Implementation: campuspaddy-ai-importer and campuspaddy-retrieval-api

License: MIT

Public API: <https://campuspaddy-ai.edjenuwahome.uk>

18 Supporting Documentation

See documentation in:

docs/00-academic-submission.md - Submission guide

docs/01-full-command-history.md - Complete setup guide

docs/02-architecture.md - Architecture overview

docs/03-api-reference.md - API reference

19 Performance Optimization

19.1 Before Optimization

Retrieval: 7-12 seconds

Ollama: 4-5 seconds

Total uncached: 13-16 seconds

19.2 After Optimization

Retrieval: 0.95 seconds (88% improvement)

Ollama: 4.3 seconds (consistent)

Total uncached: 5.3 seconds (68% improvement)

19.3 Optimization Techniques

1. Full-text search indexing with tsvector
2. Query optimization and early termination
3. Result limiting and diversification
4. Connection pooling
5. Model keep-alive in VRAM

20 System Architecture Diagram

21 Submission Evidence

Evidence is provided through the source code, database schema, API documentation, performance notes, deployment templates, and generated report files.

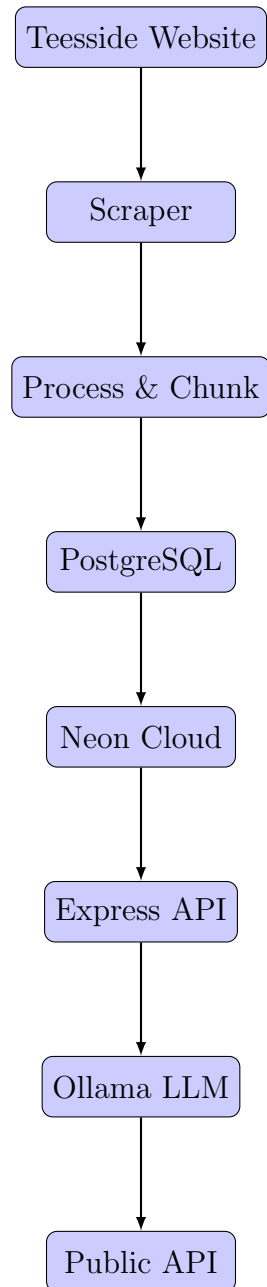


Figure 1: CampusPaddy System Pipeline